

Understanding Users and Groups in Linux

Learning Goals

- Explain why Linux systems use users and groups.
- Understand how accounts, groups, and permissions work together to secure the system.

What Is a User?

A **user account** is the foundation of Linux security. It acts as a container that defines what a person (or process) can do on the system.

Each user is identified by:

- **Username** – human-friendly label for logging in and identifying owners.
- **UID (User ID)** – a unique numeric identifier used internally by the operating system.

When you log in interactively, Linux confirms your identity using a **password** (or other authentication method). Once authenticated, everything you do — every process you launch, every file you create — is tagged with your user account.

This is what enables the system to:

- Enforce **file ownership** (who can read, write, or execute).
- Control which **processes** you can interact with.
- Maintain **security boundaries** so one user cannot freely interfere with another.

Types of User Accounts

Linux generally has three categories of users:

1. **Superuser (root)**
 - o Username: root
 - o UID: 0
 - o Has unrestricted control over the system — can modify any file, start/stop services, install software, and more.
1. **System Users**
 - o Used by background services (daemons) such as web servers, databases, or schedulers.
 - o These accounts do not normally allow interactive login.
 - o Their purpose is to isolate services from each other and from human users for security.
1. **Regular Users**
 - o The accounts that people log in with for daily work.
 - o Have limited permissions and can only modify files they own (unless explicitly granted access).

Viewing User Information

Use the `id` command to display information about your current user session:

```
[user01@host ~]$ id
uid=1000(user01) gid=1000(user01) groups=1000(user01)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

To check a different user, pass their name as an argument:

```
[user01@host ~]$ id user02
uid=1002(user02) gid=1001(user02) groups=1001(user02)
```

Working with Ownership

Each file and directory has an owner. You can verify ownership with `ls -l`:

```
[user01@host ~]$ ls -l mytextfile.txt
-rw-rw-r--. 1 user01 user01 0 Feb 5 11:10 mytextfile.txt
```

- **Column 3:** Owner username
- **Column 4:** Group owner

To check a directory itself (not its contents):

```
[user01@host ~]$ ls -ld Documents
drwxrwxr-x. 2 user01 user01 6 Feb 5 11:10 Documents
```

Viewing Running Processes

Processes also run under user accounts. Use `ps` to inspect them:

- `ps` – shows processes in the current shell.
- `ps -a` – shows all processes with a terminal.
- `ps -u` – displays the username associated with each process.

Example:

```
[user01@host ~]$ ps -au
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root       1690  0.0  0.0 220984  1052 ttyS0    Ss+  22:43   0:00 /sbin/agetty ...
user01    1769  0.0  0.1 377700  6844 tty2      Ssl+  22:45   0:00 /usr/libexec/gdm-
x-session ...
user01    1773  1.3  1.3 528948 78356 tty2      Sl+   22:45   0:03
/usr/libexec/Xorg ...
```

Even though usernames appear in the output, the kernel actually tracks them by UID behind the scenes.

The /etc/passwd File

User account data is stored in `/etc/passwd` for local accounts. Each line represents one user in this format:

```
user01:x:1000:1000:User One:/home/user01:/bin/bash
```

Field breakdown:

- **user01** – Username
- **x** – Placeholder for password (actual password hash is in `/etc/shadow`)
- **1000** – UID
- **1000** – Primary GID (group ID)
- **User One** – Comment field, usually full name or description
- **/home/user01** – Home directory
- **/bin/bash** – Login shell (or `/sbin/nologin` for disabled logins)

What Is a Group?

A **group** is a collection of users who need shared access to files or resources. Instead of giving permission to each user individually, you can grant access to a group.

Groups have:

- **Name** – human-friendly identifier.
- **GID (Group ID)** – numeric identifier.

Group information is stored in `/etc/group`. Example:

```
group01:x:10000:user01,user02,user03
```

Field breakdown:

- **group01** – Group name
- **x** – Placeholder for group password (no longer used)
- **10000** – GID
- **user01,user02,user03** – Members of this group

Primary vs Supplementary Groups

- **Primary group:** Each user has exactly one primary group (listed in `/etc/passwd`). Files they create belong to this group.
- **Supplementary groups:** Additional groups the user is a member of, stored in `/etc/group`. These provide extra access rights.

Example:

```
[user01@host ~]$ id
uid=1001(user01) gid=1003(user01) groups=1003(user01),10(wheel),10000(group01)
```

Here, user01 belongs to:

- Primary group: user01 (GID 1003)
- Supplementary groups: wheel, group01

This allows user01 to access files readable by any of these groups.

References

- `man id`, `man passwd`, `man group`
- GNU C Library Manual – Section 30: Users and Groups (requires `glibc-devel` package)

Gaining Superuser Access on Linux

Learning Objectives

- Switch to the superuser (root) account to perform system-wide administration tasks.
- Grant and manage superuser privileges for other users using `sudo`.

Understanding the Superuser

In every major operating system, there is a **superuser** account with unrestricted control over the system.

On Linux (including Red Hat Enterprise Linux), this account is named **root** and has a **UID of 0**.

The root user can:

- Install or remove software.
- Edit system files and configuration.
- Start and stop services.
- Manage user accounts and permissions.
- Access or modify any file on the system.

This power is essential for system administration but also **dangerous** — root can accidentally (or maliciously) delete critical files, break the system, or create security backdoors.

Because of these risks, Linux best practice is:

- **Log in as a normal user for daily work.**
- **Elevate privileges only when required**, using tools like `su`, `sudo`, or `PolicyKit`.

Note: This is similar to modern Windows systems, where administrators typically log in as a standard user and rely on UAC or membership in the Administrators group to perform privileged actions.

Switching to Another User

You can switch accounts with the su command:

```
[user01@host ~]$ su - user02
Password: user02_password
[user02@host ~]$
```

Key points:

- Running `su - username` starts a **login shell** for that user, loading their full environment.
- Running `su username` (without `-`) starts a shell but keeps the current user's environment.
- Running `su -` without specifying a user switches to root.

If you are already root, you can switch users without entering their password.

Tip: Administrators usually prefer `su -` to ensure they get the correct root environment variables.

You can also run a **single command** as another user with:

```
su -c "command" username
```

This is similar to Windows' `runas` utility.

Running Commands with sudo

In many modern systems, direct root login is disabled or root's password is locked for security reasons.

Instead of logging in as root, you use `sudo` to run commands with elevated privileges.

Key Differences Between `su`, `su -`, and `sudo`

Command	Becomes	Environment Used	Password Needed	Privileges	Logging
<code>su</code>	New user	Current user's	Target user's	Full shell as target user	Only <code>su</code> use is logged
<code>su -</code>	New user	Target user's	Target user's	Full login shell	Only <code>su</code> use is logged
<code>sudo</code>	Per-command	Current user's	Your own	Limited to allowed commands	Every command logged

With `sudo`:

- You authenticate using **your own password**, not root's.
- Access is controlled by rules in `/etc/sudoers`.
- Every command you run is logged to `/var/log/secure`.

Example:

```
[user01@host ~]$ sudo usermod -L user02
[sudo] password for user01: .....
```

If a user is not allowed to run a command:

```
[user02@host ~]$ sudo tail /var/log/secure
user02 is not in the sudoers file. This incident will be reported.
```

This failed attempt is also logged for auditing.

Getting an Interactive Root Shell

If you want a root shell (not just a single command):

- `sudo -i` – Opens an interactive root login shell, loading root's environment.
- `sudo -s` – Opens a root shell without sourcing login scripts.

Example on a cloud instance:
[ec2-user@host ~]\$ sudo -i
[root@host ~]#

Configuring sudo Access

The configuration file for sudo is /etc/sudoers.

You should always edit it with visudo to prevent syntax errors and conflicts if multiple admins edit it simultaneously.

Example: Grant wheel Group Full Root Access

```
%wheel ALL=(ALL:ALL) ALL
```

Explanation:

- %wheel → Applies to all users in the wheel group.
- ALL=(ALL:ALL) → They can run commands as any user and group.
- ALL → They can run any command.

Adding Per-User or Per-Group Access

You can drop custom files in /etc/sudoers.d/ to grant access without editing the main file.

Full access for a single user:

```
user01 ALL=(ALL) ALL
```

Full access for a group:

```
%group01 ALL=(ALL) ALL
```

Run one specific command as another user:

```
%games ALL=(operator) /bin/id
```

Passwordless sudo

For automation or cloud servers, you can allow passwordless sudo:

```
ansible ALL=(ALL) NOPASSWD: ALL
```

⚠ **Warning:** This bypasses password checks, so use it only for tightly controlled accounts (e.g., with SSH key authentication).

Example: AWS EC2 instances ship with:

```
ec2-user ALL=(ALL) NOPASSWD: ALL
```

allowing ec2-user to run any command as root without a password.

References

- man su, man sudo, man visudo, man sudoers
- GNU C Library Manual – Section 30.2: *Persona of a Process*

Managing Local User Accounts

Learning Objectives

- Create new user accounts.
- Modify existing user accounts.

- Safely delete or disable user accounts.

Managing Local Users

Linux provides several command-line utilities for managing user accounts. These tools let administrators add, update, lock, unlock, and remove users quickly.

Creating Users

The `useradd` command creates a new account:
`useradd username`

This command:

- Creates the account entry in `/etc/passwd`.
- Sets up a home directory (by default `/home/username`).
- Creates a **user private group** with the same name as the user.

At this stage, no password is set — the account is disabled until a password is assigned.

You can display available options for `useradd` with:
`useradd --help`

Defaults and UID Assignment

- Defaults for newly created accounts are controlled by `/etc/login.defs`.
- These defaults include UID ranges, password aging rules, and other policies.
- On RHEL 9 and similar systems, `useradd` assigns the first available UID ≥ 1000 , unless you specify one manually using `-u`.

Modifying Users

You can modify existing accounts with the `usermod` command. View available options:
`usermod --help`

Common usermod Options

Option	Purpose
<code>-a, --append</code>	Add user to supplementary groups (used with <code>-G</code>).
<code>-c, --comment COMMENT</code>	Add or change the GECOS/comment field (e.g., full name).
<code>-d, --home HOME_DIR</code>	Set a new home directory path.
<code>-g, --gid GROUP</code>	Change the user's primary group.
<code>-G, --groups GROUPS</code>	Define supplementary groups (comma-separated).
<code>-L, --lock</code>	Lock the user account (disables login).
<code>-m, --move-home</code>	Move user's existing home to a new location (used with <code>-d</code>).
<code>-s, --shell SHELL</code>	Set a new default login shell.
<code>-U, --unlock</code>	Unlock a previously locked account.

Deleting Users

Use `userdel` to remove an account:
`userdel username`

This removes the account entry from `/etc/passwd` but leaves the home directory and files intact.
 To remove both the account and its home directory:

```
userdel -r username
```

Security Risk of Orphaned Files

If you delete a user without `-r`, their files remain owned by their old UID.

If a new user is later created with the same UID, they inherit ownership of these files — which can leak sensitive data.

Example:

```
[root@host ~]# useradd user01
[root@host ~]# userdel user01
[root@host ~]# useradd -u 1000 user02
```

Now `user02` owns all files that `user01` previously owned.

To find files without owners or groups:

```
find / -nouser -o -nogroup
```

Many organizations prefer **locking accounts** instead of deleting them to avoid this risk.

Setting Passwords

Use `passwd` to set or change a user's password:

```
passwd username
```

Example:

```
[root@host ~]# passwd user01
Changing password for user user01.
New password: redhat
BAD PASSWORD: The password is shorter than 8 characters
Retype new password: redhat
passwd: all authentication tokens updated successfully.
```

Key points:

- The root user can set any password (even weak ones).
- Normal users must choose a password meeting minimum system policy (length, complexity, not equal to username).

UID Ranges

Linux assigns UIDs based on purpose:

UID Range	Purpose
0	Root (superuser).
1–200	Static system accounts (reserved for system services).
201–999	Dynamically allocated system accounts for daemons without file ownership needs.
1000+	Regular human user accounts.

Older systems (RHEL 6 and earlier) used 1–499 for system users and 500+ for regular users.

You can adjust UID allocation ranges in `/etc/login.defs`.

References

- `man useradd`
- `man usermod`
- `man userdel`

Managing Local Group Accounts

Objectives

- Create, rename, and delete local groups.
- Modify group membership for users.

Group Management Overview

Linux offers several command-line tools for creating, editing, and removing groups. While graphical tools exist, most administrators prefer using the terminal for precision and automation.

Creating Groups

Use the `groupadd` command to create a new group:

```
groupadd group01
```

By default:

- The system picks the next available GID within the range specified in `/etc/login.defs` (`GID_MIN` to `GID_MAX`).
- If lower unused GIDs exist, they are not reused automatically — the highest available value is selected.

To manually specify a GID:

```
groupadd -g 10000 group01
```

Result in `/etc/group`:

```
group01:x:10000:
```

To create a **system group**, use `-r`:

```
groupadd -r group02
```

System groups are assigned GIDs from the `SYS_GID_MIN`–`SYS_GID_MAX` range in `/etc/login.defs`.

Modifying Existing Groups

Use `groupmod` to rename a group or change its GID.

Rename a group:

```
groupmod -n group0022 group02
```

Change a group's GID:

```
groupmod -g 20000 group0022
```

Both the group name and GID are updated in `/etc/group` immediately.

Deleting Groups

Remove a group with:

```
groupdel group0022
```

You cannot delete a group if it is set as a user's primary group. First reassign the user to a different primary group or delete the user account.

Managing Group Membership

Group membership is controlled with `usermod`.

Change a User's Primary Group

```
usermod -g group01 user02
```

Before:

```
uid=1006(user02) gid=1008(user02) groups=1008(user02)
```

After:

```
uid=1006(user02) gid=10000(group01) groups=10000(group01)
```

Add a User to a Supplementary Group

```
usermod -aG group01 user03
```

Before:

```
uid=1007(user03) gid=1009(user03) groups=1009(user03)
```

After:

```
uid=1007(user03) gid=1009(user03) groups=1009(user03),10000(group01)
```

Important: Always include the `-a` (append) flag when using `-G`, otherwise the user is removed from all other supplementary groups not listed in the command.

Primary vs. Supplementary Groups

- **Primary group:** Defined in `/etc/passwd` and used as the default group for new file creation. A user can belong to only one primary group at a time.
- **Supplementary groups:** Additional memberships listed in `/etc/group`. A user can be in multiple supplementary groups.

Access control is identical whether a group is primary or supplementary — the difference only affects which group owns newly created files.

Temporarily Changing Primary Group

You can switch your active primary group for the current shell session with `newgrp`.

Example:

```
[user03@host ~]$ id
uid=1007(user03) gid=1009(user03) groups=1009(user03),10000(group01)
```

```
[user03@host ~]$ newgrp group01
```

```
[user03@host ~]$ id
uid=1007(user03) gid=10000(group01) groups=1009(user03),10000(group01)
```

This is useful when creating files that should belong to a different group. The change lasts until you log out.

References

- `man group`
- `man groupadd`
- `man groupdel`
- `man usermod`

Managing User Passwords

Objectives

- Define and enforce password management policies for users.
- Lock and unlock user accounts manually when required.

Shadow Passwords and Security

Historically, password hashes were stored in `/etc/passwd`, a file readable by all users. As attacks became more sophisticated (especially dictionary attacks), Linux moved password hashes to `/etc/shadow`, which is readable only by the root user.

Each user account has an entry in `/etc/shadow` with **nine colon-separated fields**.

Example:

```
user03:$6$CSsXsd3rwghsdfarf:17933:0:99999:7:2:18113:
```

Breakdown of fields:

- **user03** – Username.
- **\$6\$CSsXsd3rwghsdfarf** – Hashed password, including algorithm, salt, and hash.
- **17933** – Date of last password change, counted as days since 1970-01-01 UTC.
- **0** – Minimum number of days before password can be changed again.
- **99999** – Maximum password age. Empty means it never expires.
- **7** – Number of days before expiration to warn the user.
- **2** – Days of inactivity allowed after expiration before account is locked.
- **18113** – Account expiration date, in days since epoch. Empty means no expiration.
- **(last field)** – Reserved for future use.

Structure of a Hashed Password

The hashed password field contains three elements, separated by \$:

```
$6$CSsXcYG1L/4ZfHr/$2W6evvJahUfzfHpc9X.45Jc6H30E
```

- **6** – Hash algorithm (6 = SHA-512, 5 = SHA-256, 1 = MD5).
- **CSsXcYG1L/4ZfHr/** – Salt (random data unique to each password).
- **2W6evvJahUfzfHpc9X.45Jc6H30E** – The resulting hash of the salted password.

Salting passwords ensures that even if two users have the same password, their hashes are different, making precomputed hash attacks ineffective.

Hash anatomy (field 2) — how to read \$id\$salt\$hash

- \$y is associated with **yescrypt**, which is a modern password-hashing scheme designed to be more resistant to attacks (derived from the scrypt family) — this is documented in `crypt`
- \$6\$...\$... → **SHA-512** (modern default on many distros)
- \$5\$...\$... → **SHA-256**
- \$1\$...\$... → **MD5** (legacy, avoid)
- Middle part is the **salt** (random, per-user); last part is the **hash**.
- Salt means two users with the same password still get **different hashes**, killing rainbow-table reuse.

Password Verification

When a user attempts to log in:

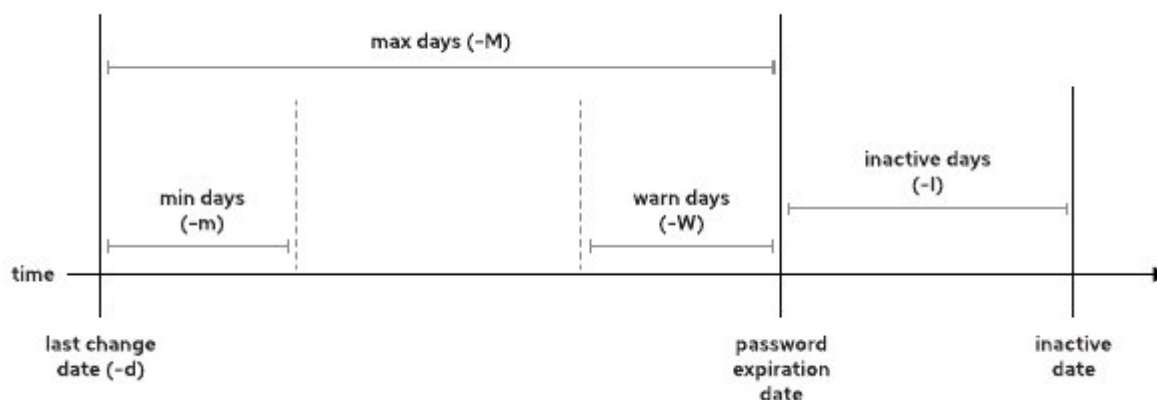
1. The system retrieves their `/etc/shadow` entry.
2. The provided password is combined with the stored salt.
3. The result is hashed with the specified algorithm.
4. If the output matches the stored hash, authentication succeeds.

This method allows verification without storing the plaintext password.

Configuring Password Aging

Password aging policies are managed with the `chage` command (**change age**).

Example:



```
chage -m 0 -M 90 -W 7 -I 14 sysadmin05
```

This sets:

- **Minimum age** -m 0 – user can change password immediately.
- **Maximum age** -M 90 – password must be changed every 90 days.
- **Warning period** -W 7 – warn 7 days before expiration.
- **Inactivity period** -I 14 – lock account after 14 days of inactivity post-expiration.

Setting Account Expiration

Example for expiring a user account 30 days from today:

```
date +%F
date -d "+30 days" +%F
chage -E $(date -d "+30 days" +%F) cloudadmin10
chage -l cloudadmin10 | grep "Account expires"
```

- -E sets account expiration date.
- -l displays the user's password aging information.

Forcing Password Change at Next Login

If you need a user to reset their password:

```
chage -d 0 cloudadmin10
```

The system will prompt for a new password at the next login.

Default Password Policy

Global defaults are stored in `/etc/login.defs`:

- **PASS_MAX_DAYS** – Maximum password age.

- **PASS_MIN_DAYS** – Minimum password age.
- **PASS_WARN_AGE** – Warning period before expiration.

Changes affect new accounts only. Existing users keep their current policies unless modified manually.

Locking and Unlocking Accounts

Use `usermod` to disable or re-enable logins:

```
usermod -L sysadmin03    # Lock account
usermod -U sysadmin03    # Unlock account
```

You can combine locking with an expiration date:

```
usermod -L -e 2022-08-14 cloudadmin10
```

This prevents password logins and enforces account expiration.

Using the `no login` Shell

For accounts that should never log in interactively, assign `/sbin/nologin` as the shell:

```
usermod -s /sbin/nologin newapp
```

Attempting to log in will display:

```
This account is currently not available.
```

This is common for service accounts that are needed by applications but not by human users.

Note: `nologin` prevents interactive login but does not block all forms of authentication. Programs such as mail clients or file transfer tools may still work if they rely on the user's password.

References

- `man chage`
- `man usermod`
- `man shadow`
- `man crypt`
- `man login.defs`

Adjusting Tuning Profiles

Objectives

- Improve system performance by selecting and managing tuning profiles using the **tuned** daemon.

System Tuning Overview

System performance can be optimized by adjusting device parameters for different workload types. The **tuned** daemon applies these optimizations automatically, using predefined profiles tailored for specific scenarios.

Static Tuning

Static tuning applies a set of kernel and system parameters when the **tuned** service starts or when a new profile is selected. These parameters stay in effect until changed, providing consistent performance settings regardless of system activity.

Dynamic Tuning

Dynamic tuning adapts system settings in real time based on current workload. The tuned daemon continuously monitors components such as CPU, storage, and network, then adjusts parameters to improve performance under load or reduce power usage when idle.

Examples:

- **Storage devices** are heavily used during system startup but mostly idle afterward.
- **CPU and network** activity spikes during peak hours and drops during idle times.

Dynamic tuning ensures performance is optimized automatically during these changes.

Monitor and Tuning Plug-ins

The tuned daemon uses:

- **Monitor plug-ins** – Collect system data (e.g., CPU load, disk I/O, network traffic).
- **Tuning plug-ins** – Apply settings to subsystems based on the collected data.

Monitor plug-ins available by default:

- **disk** – Tracks disk I/O operations.
- **net** – Measures network packet activity.
- **load** – Observes CPU utilization.

Tuning plug-ins examples:

- **disk** – Adjusts disk scheduler, power management, spin-down settings.
- **net** – Configures network speed and Wake-on-LAN.
- **cpu** – Sets CPU governor and latency tuning.

Dynamic tuning is disabled by default. Enable it by editing `/etc/tuned/tuned-main.conf`:

```
dynamic_tuning = 1
update_interval = 10
```

`update_interval` defines how often the daemon checks and adjusts parameters (in seconds).

Installing and Enabling tuned

If not installed, add and enable tuned with:

```
dnf install tuned
systemctl enable --now tuned
```

Tuning Profile Categories

Tuned provides two main profile types:

- **Power-saving profiles** – Reduce energy consumption.
- **Performance profiles** – Maximize throughput or reduce latency.

Performance profiles focus on:

- Storage and network throughput.
- Low-latency workloads.
- Virtual machine performance.
- Virtualization host optimization.

Profiles in RHEL 9 & 10

Profile	Purpose
balanced	Default; balance between power savings and performance.
powersave	Aggressive power-saving mode.
throughput-performance	Maximizes I/O and CPU throughput.
accelerator-performance	Similar to throughput-performance with <100µs latency.
latency-performance	Prioritizes deterministic low latency over power savings.
network-throughput	Adds extra network tuning to throughput-performance.
network-latency	Low-latency network tuning based on latency-performance.
desktop	Optimizes for interactive responsiveness.
hpc-compute	Ideal for high-performance computing workloads.
virtual-guest	Optimizes for virtual machines.
virtual-host	Optimizes for virtualization hosts.
intel-sst	Enhances Intel Speed Select Technology setups.
optimize-serial-console	Improves console responsiveness; used as overlay.

Profile Storage

Profiles are stored in:

- `/usr/lib/tuned` – Default system profiles.
- `/etc/tuned` – Custom profiles (override system profiles with same name).

Each profile directory contains a `tuned.conf` configuration file and optional supporting files.

Example:

```
cd /usr/lib/tuned/virtual-guest
cat tuned.conf
```

This file might include:

- `[main]` section with a summary and optional `include` directive to inherit another profile.
- `[sysctl]` section to set kernel parameters like `vm.dirty_ratio` or `vm.swappiness`.

When creating a custom profile, copy it to `/etc/tuned/` and modify it there to avoid overwriting system files during package updates.

Managing Profiles from the Command Line

View Current Profile

```
tuned-adm active
```

List Available Profiles

```
tuned-adm list
```

Displays all system and custom profiles, marking the active one.

View Profile Details

```
tuned-adm profile_info network-latency
```

If no profile name is specified, it shows details for the currently active profile.

Change Active Profile

```
tuned-adm profile throughput-performance
```

Verify with:

```
tuned-adm active
```

Get Profile Recommendation

```
tuned-adm recommend
```

The recommended profile is based on hardware characteristics and installation choices.

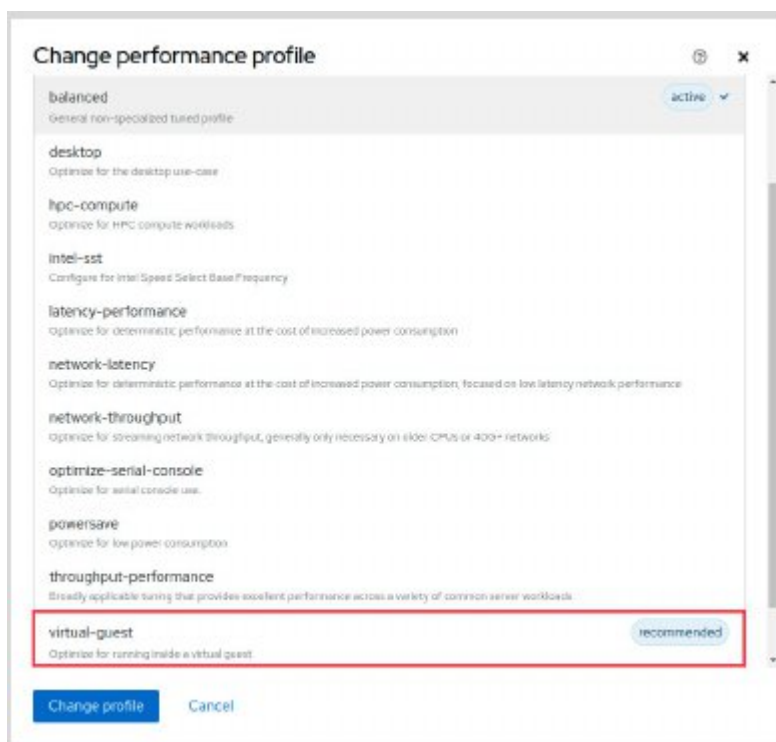
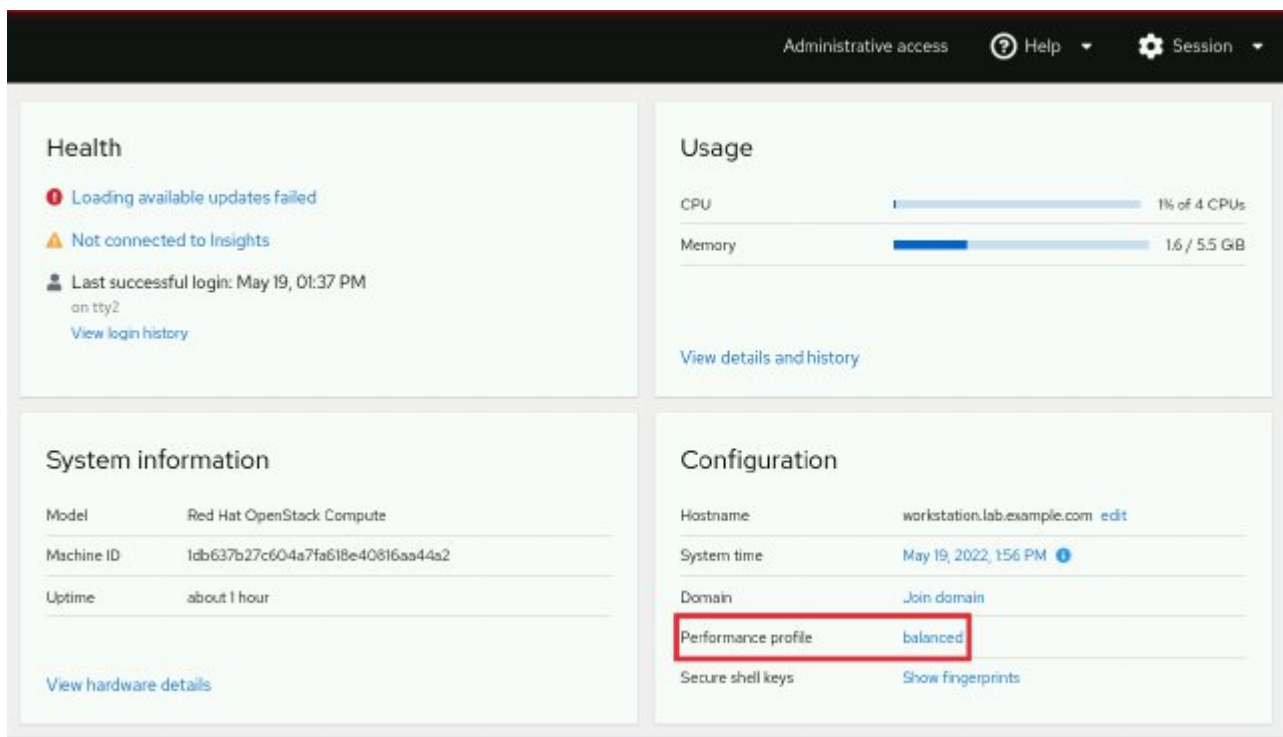
Disable Tuning

```
tuned-adm off
```

This removes any active profile until one is manually re-enabled.

Managing Profiles in the Web Console

1. Log in to the web console and escalate to administrative mode.
2. Go to **Overview** → **Performance profile**.
3. Select a new profile from the list and confirm with **Change profile**.
4. Return to Overview to verify the change.



Always switch back to limited access after completing administrative tasks for security reasons.

References

- man tuned
- man tuned.conf
- man tuned-main.conf
- man tuned-adm
- Red Hat Enterprise Linux 9 – *Monitoring and Managing System Status and Performance* Guide

Influence Process Scheduling

Objectives

- Raise or lower the CPU priority of processes using `nice` and `renice`.

Linux Scheduling in Brief

Linux divides CPU time among runnable threads through **time-slicing** (multitasking). The scheduler rapidly rotates threads across cores so many programs appear to run at once. On light loads, everything runs immediately; under heavy CPU pressure, the scheduler must decide which threads run first and how long.

Scheduling Policies and CFS

Linux supports several policies. Real-time classes (e.g., `SCHED_FIFO`, `SCHED_RR`) obey strict priorities. Normal, non-real-time work uses the **Completely Fair Scheduler (CFS)** under `SCHED_NORMAL`/`SCHED_OTHER`.

- CFS keeps runnable tasks in a time-weighted tree keyed to how much CPU they've already consumed: the task that has used **least** CPU gets scheduled **next**.
- Normal tasks have a fixed real-time priority of 0 (so any real-time task out-ranks them), but CFS's selection is driven by virtual runtime rather than that static value.

Nice Values

Each process has a **nice** value from **-20** (higher priority) to **19** (lower priority); default is **0**.

- Lower (more negative) nice $\Rightarrow$ runs **sooner/more often**.
- Higher nice $\Rightarrow$ runs **later/less often** (i.e., "nicer" to others).
- Children inherit the parent's nice value.
- Any user may **increase** a process's nice (lower its importance). Only root may **decrease** it (raise importance).

On an idle system, priority hardly matters—everything runs promptly. On a saturated system, lower nice values get CPU first, but CFS still rotates time fairly based on past CPU usage.

Who May Change Nice?

- **Privileged (root):** can reduce nice values (raise priority) for any process.
- **Unprivileged users:** can only raise nice values (lower priority) on their own processes; they cannot alter other users' tasks or make their own tasks less nice than they already are.

Seeing Priority and Nice

`top` shows:

- **PR** column = kernel priority (mapped value)

- **NI** column = nice value

Real-time tasks display special PR values; normal tasks show PR around 20 with NI reflecting the user-visible adjustment. Example snippet:

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	172180	16232	10328	S	0.0	0.3	0:01.49	systemd
3	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_gp

ps can print nice and class information:

```
ps axo pid,comm,nice,cls --sort=-nice
```

- CLS shows TS (time sharing = normal CFS), FF (FIFO), RR (round-robin).
- Real-time processes do **not** have nice values (NI appears as -).



Start a Process with a Specific Nice

New processes inherit the shell's nice (commonly 0). Launch with nice to run at lower priority:

```
sleep 60 & # inherits NI=0
ps -o pid,comm,nice $! # shows NI for the last background PID
```

```
nice sleep 60 & # default adds +10 ⇒ NI=10
ps -o pid,comm,nice $!
```

```
nice -n 15 sleep 60 & # explicit NI=15
ps -o pid,comm,nice $!
```

Using a higher NI makes the job less intrusive to interactive work.

Change Nice of an Existing Process

Adjust a running task with renice (PID shown as 2740 here):

```
renice -n 19 2740
# 2740 (process ID) old priority 15, new priority 19
```

In **top**, press **r** to renice interactively: enter the PID, then the new NI.

References

- man nice
- man renice
- man top
- man sched_setscheduler